

Algoritmos Avançados

Aula 03: Tipos Abstratos de Dados (TADs) & Listas em Python

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário

- 1 Conceito de Tipo Abstrato de Dados (TAD)
- 2 Estrutura Interna das Listas em Python
- 3 Criação, Indexação e Fatiamento (Slicing)
- 4 Métodos Nativos e Operações do TAD Lista
- 5 List Comprehensions e Performance
- 6 Exercícios Práticos Resolvidos

O que é um Tipo Abstrato de Dados (TAD)?

Definition (Tipo Abstrato de Dados)

Um **TAD** é um modelo matemático para tipos de dados definido pelo seu **comportamento** (semântica) do ponto de vista do usuário, especificando o que o dado faz, e não como ele é implementado internamente.

Interface (O Que Faz)

- Conjunto de operações permitidas.
- Contrato de uso do dado.
- Esconde detalhes de baixo nível.

Implementação (Como Faz)

- Estrutura de memória concreta.
- Algoritmos internos de manipulação.
- Pode ser alterada sem quebrar o código do cliente.

Por que estudar TADs em Algoritmos Avançados?

- **Encapsulamento e Abstração:** Permite resolver problemas complexos tratando estruturas como "caixas-pretas" confiáveis.
- **Reutilização e Manutenibilidade:** A alteração da estrutura interna (ex: trocar lista encadeada por vetor dinâmico) não impacta a aplicação.
- **Base para Estruturas Complexas:** Grafos, Árvores, Filas de Prioridade, Pilhas e Filas são todos conceituados primeiramente como TADs.

Analogia do Mundo Real

Ao dirigir um carro, você interage com a interface (volante, pedais, marcha) sem precisar modificar diretamente a injeção eletrônica ou o pistão do motor.

O TAD Lista: Definição Formal

Conceito

Uma **Lista** é uma sequência finita e ordenada de elementos do mesmo tipo (ou de tipos heterogêneos), representada formalmente como:

$$L = (a_1, a_2, a_3, \dots, a_n)$$

onde n é o tamanho da lista e a_i é o i -ésimo elemento.

Operações Básicas de qualquer TAD Lista

- **Criar/Inicializar:** Cria uma lista vazia.
- **Inserir(x, i):** Adiciona o elemento x na posição i .
- **Remover(i):** Remove e retorna o elemento da posição i .
- **Acessar(i):** Retorna o valor contido na posição i .
- **Tamanho():** Retorna a quantidade n de elementos.

Como o Python Implementa o TAD Lista?

No Python, a classe nativa `list` não é uma lista encadeada (Linked List), mas sim um Vetor Dinâmico (*Dynamic Array / Redimensionável*).

Características da Implementação

- Bloco contíguo de memória contendo ponteiros para os objetos.
- Acesso direto por índice via cálculo de endereço em tempo constante $O(1)$.
- Permite coleções heterogêneas (misturando números, textos, objetos).

Estratégia de Redimensionamento

- Aloca memória extra antecipadamente (*over-allocation*).
- Quando o limite estoura, aloca um bloco maior ($\approx 1.125 \times$ tamanho antigo) e copia os ponteiros.
- Garante custo amortizado $O(1)$ para a operação `append()`.

Visualizando a Lista em Memória

Uma lista armazena referências para os objetos na memória, não os valores em si.

```
1 # Lista contendo elementos de diferentes tipos
2 dados = [42, "Agro", True, 3.14]
3
4 # Estrutura na Memoria (Representacao Conceitual):
5 # dados -> [ Pointer_0, Pointer_1, Pointer_2, Pointer_3 ]
6 #           |           |           |           |
7 #           v           v           v           v
8 #       (int: 42) ("Agro")  (bool: T)  (float: 3.14)
```

Consequência de Desempenho

O acesso `dados[i]` é ultra-rápido $O(1)$, mas inserir/remover elementos no meio ou início requer mover todos os ponteiros subsequentes ($O(n)$).

Formas de Criar e Inicializar Listas

```
1 # 1. Utilizando sintaxe literal com colchetes []
2 lista_vazia = []
3 frutas = ["Laranja", "Cafe", "Milho", "Soja"]
4
5 # 2. Utilizando o construtor list()
6 numeros = list(range(1, 6))          # [1, 2, 3, 4, 5]
7 caracteres = list("FATEC")           # ['F', 'A', 'T', 'E', 'C']
8
9 # 3. Inicializacao com repeticao de tamanho fixo
10 zeros = [0] * 5                      # [0, 0, 0, 0, 0]
11 matriz_linha = [None] * 4            # [None, None, None, None]
```

Indexação Positiva e Negativa

Os elementos são acessados através de índices inteiros baseados em zero.

```
1 talhoes = ["T-Alfa", "T-Beta", "T-Gama", "T-Delta"]
2
3 # Indexacao Positiva (do inicio para o fim: 0 a n-1)
4 print(talhoes[0])    # T-Alfa
5 print(talhoes[2])    # T-Gama
6
7 # Indexacao Negativa (do fim para o inicio: -1 a -n)
8 print(talhoes[-1])   # T-Delta (ultimo elemento)
9 print(talhoes[-2])   # T-Gama (penultimo elemento)
10
11 # Tentar acessar indice inexistente lanca IndexError
12 # print(talhoes[10]) # IndexError: list index out of range
```

Fatiamento de Listas (Slicing)

A sintaxe de fatiamento permite extrair sublistas: `lista[inicio:fim_exclusivo:passo]`

```
1 valores = [10, 20, 30, 40, 50, 60, 70, 80]
2
3 # Sublista do indice 2 ao 4 (fim e exclusivo)
4 print(valores[2:5])      # [30, 40, 50]
5
6 # Omite inicio (comeca em 0) ou omite fim (vai ate o final)
7 print(valores[:3])      # [10, 20, 30]
8 print(valores[5:])      # [60, 70, 80]
9
10 # Fatiamento com passo
11 print(valores[::2])     # [10, 30, 50, 70] (elementos nas posicoes pares)
12 print(valores[::-1])    # [80, 70, 60, 50, 40, 30, 20, 10] (Inversao
    completa)
```

Modificação e Alteração via Slicing

Por ser uma estrutura mutável, o fatiamento pode ser usado para substituir, inserir ou deletar intervalos de elementos diretamente.

```
1 letras = ['a', 'b', 'c', 'd', 'e']
2
3 # Substituindo uma fatia por outra lista
4 letras[1:4] = ['B', 'C', 'D']
5 print(letras)  # ['a', 'B', 'C', 'D', 'e']
6
7 # Inserindo elementos em um ponto específico sem apagar nada
8 letras[1:1] = ['X', 'Y']
9 print(letras)  # ['a', 'X', 'Y', 'B', 'C', 'D', 'e']
10
11 # Removendo uma fatia
12 letras[1:3] = []
13 print(letras)  # ['a', 'B', 'C', 'D', 'e']
```

Atribuição de Referência vs. Cópia de Lista

Atribuir uma lista existente a uma nova variável não cria uma nova lista, apenas compartilha a mesma referência na memória!

```
1 original = [1, 2, 3]
2 copia_falsa = original    # Ambas apontam para o MESMO objeto!
3
4 copia_falsa.append(99)
5 print(original)           # [1, 2, 3, 99] - A lista original foi alterada!
6
7 # Como fazer uma Cópia Rasa (Shallow Copy) real:
8 copia_real_1 = original.copy()
9 copia_real_2 = original[:]
10 copia_real_3 = list(original)
11
12 copia_real_1.append(100)
13 print(original)           # [1, 2, 3, 99] (Inalterada!)
14 print(copia_real_1)       # [1, 2, 3, 99, 100]
```

Cópia Rasa (Shallow) vs. Cópia Profunda (Deep Copy)

Para listas aninhadas (matrizes/listas de listas), a cópia rasa copia apenas os ponteiros do primeiro nível!

```
1 import copy
2
3 matriz = [[1, 2], [3, 4]]
4
5 # Cópia Rasa (Shallow Copy)
6 rasa = matriz.copy()
7 rasa[0][0] = 999
8 print(matriz)    # [[999, 2], [3, 4]] -> O nível interno foi modificado!
9
10 # Cópia Profunda (Deep Copy) - Duplica toda a árvore de objetos
11 matriz_original = [[1, 2], [3, 4]]
12 profunda = copy.deepcopy(matriz_original)
13 profunda[0][0] = 999
14 print(matriz_original)    # [[1, 2], [3, 4]] -> Totalmente preservada!
```

Adição e Inserção de Elementos

`append(x)`

Adiciona o elemento x ao final da lista.

- Complexidade: $O(1)$ amortizado.

`insert(i, x)`

Insere x na posição de índice i .

- Complexidade: $O(n)$ (desloca elementos à direita).

`extend(iterable)`

Adiciona todos os itens de um iterável ao final da lista.

- Complexidade: $O(k)$ (k = itens adicionados).

```
1 l = [10, 20]
2 l.append(30)           # [10, 20, 30]
3 l.insert(0, 5)         # [5, 10, 20, 30]
4 l.extend([40, 50])     # [5, 10, 20, 30, 40, 50]
```

Concatenação: + vs. += vs. extend()

Atenção ao desempenho e reatribuição de memória ao juntar listas!

```
1 a = [1, 2]
2 b = [3, 4]
3
4 # Operador + (Cria um NOVO objeto lista na memoria) - O(n + m)
5 c = a + b
6 print(c) # [1, 2, 3, 4]
7
8 # Operador += (Modifica a lista 'a' IN-PLACE) - O(m)
9 a += b # Equivalente a a.extend(b)
10 print(a) # [1, 2, 3, 4]
```

Boa Prática

Para modificar uma lista existente adicionando múltiplos elementos, prefira `extend()` ou `+=` ao invés de `a = a + b`.

Remoção de Elementos

Formas de Remover Elementos

- `pop([i])`: Remove e retorna o elemento no índice i (padrão $i = -1$, último item).
- `remove(x)`: Remove a primeira ocorrência do valor x . Se x não existir, lança `ValueError`.
- `clear()`: Remove todos os elementos, deixando a lista vazia (`[]`).
- Instrução `del lista[i]`: Remove o item ou fatia sem retornar valor.

```
1 frutas = ["maca", "banana", "laranja", "banana"]
2
3 ultimo = frutas.pop()           # Remove 'banana' do final - O(1)
4 frutas.remove("banana")         # Remove a 1a 'banana' encontrada - O(n)
5 del frutas[0]                   # Remove 'maca' no índice 0 - O(n)
6 print(frutas)                  # ['laranja']
```

Busca e Consulta de Elementos

```
1 codigos = [101, 204, 305, 204, 500]
2
3 # 1. Verificacao de existencia com operador 'in' - O(n)
4 if 305 in codigos:
5     print("Codigo encontrado!")
6
7 # 2. Obter o indice da PRIMEIRA ocorrencia com index() - O(n)
8 posicao = codigos.index(204)
9 print(f"O codigo 204 aparece primeiro no indice: {posicao}") # 1
10
11 # Pode-se delimitar a busca: index(valor, inicio, fim)
12 posicao_segunda = codigos.index(204, 2)
13 print(f"Segunda ocorrencia no indice: {posicao_segunda}") # 3
14
15 # 3. Contar frequencia com count() - O(n)
16 qtd = codigos.count(204)
17 print(f"Quantidade de vezes que 204 aparece: {qtd}") # 2
```

Ordenação de Listas: `sort()` vs. `sorted()`

`lista.sort()`

- Ordena a lista in-place (modifica a própria lista).
- Retorna `None`.
- Algoritmo interno: Timsort $O(n \log n)$.

`sorted(iterable)`

- Não altera a lista original.
- Retorna uma nova lista ordenada.
- Funciona para qualquer iterável.

```
1 nums = [42, 10, 5, 99, 23]
2
3 # Usando sorted()
4 ordenada = sorted(nums)
5 print(nums)          # [42, 10, 5, 99, 23] (Original mantida!)
6 print(ordenada)      # [5, 10, 23, 42, 99]
7
8 # Usando sort() in-place com ordem decrescente
9 nums.sort(reverse=True)
10 print(nums)          # [99, 42, 23, 10, 5]
```

Ordenação Customizada com a Função key

O parâmetro `key` aceita uma função de transformação aplicada a cada elemento antes da comparação.

```
1 # Ordenando strings pelo tamanho (comprimento)
2 palavras = ["Agronomia", "Soja", "Trator", "Bebedouro"]
3 palavras.sort(key=len)
4 print(palavras)    # ['Soja', 'Trator', 'Agronomia', 'Bebedouro']
5
6 # Ordenando uma lista de pares pelo segundo elemento
7 dados_talhoes = [("T-01", 120.5), ("T-02", 45.0), ("T-03", 89.2)]
8
9 # Ordena pela area (float) de forma crescente
10 dados_talhoes.sort(key=lambda item: item[1])
11 print(dados_talhoes)    # [('T-02', 45.0), ('T-03', 89.2), ('T-01', 120.5)]
```

O que são List Comprehensions?

List Comprehension é uma sintaxe concisa e fluente para criar novas listas a partir de iteráveis existentes.

Sintaxe Geral

```
[ expressao for item in iteravel if condicao ]
```

```
1 # Forma Tradicional (Com la o for)
2 quadrados_pares = []
3 for x in range(10):
4     if x % 2 == 0:
5         quadrados_pares.append(x  2)
6
7 # Forma Elegante e Pyth nica (List Comprehension)
8 quadrados_pares_py = [x  2 for x in range(10) if x % 2 == 0]
9
10 print(quadrados_pares_py)  # [0, 4, 16, 36, 64]
```

List Comprehensions Avançadas & Comportamento

```
1 # 1. Aplicando transformacoes e sanitizacao de Strings
2 nomes_brutos = [" soja ", " MILHO ", "cafe\n"]
3 limpos = [nome.strip().capitalize() for nome in nomes_brutos]
4 print(limpos) # ['Soja', 'Milho', 'Cafe']
5
6 # 2. Uso de Expressao Condicional Ternaria no valor retornado
7 # Sintaxe: [val_if if cond else val_else for item in iteravel]
8 produtividades = [72.5, 45.0, 88.0, 52.3]
9 classificacao = ["Alta" if p >= 60.0 else "Baixa" for p in produtividades]
10 print(classificacao) # ['Alta', 'Baixa', 'Alta', 'Baixa']
```

Comprehensions Aninhadas (Matrizes 2D)

```
1 # Criando uma Matriz 3x3 preenchida com zeros
2 # ATENCAO: Nao use [[0]*3]*3 (isso cria 3 referencias a MESMA linha na
   memoria!)
3 matriz_zeros = [[0 for _ in range(3)] for _ in range(3)]
4
5 # Aplatando (Flattening) uma matriz 2D em uma lista 1D
6 matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
7 vetor_1d = [elemento for linha in matriz for elemento in linha]
8 print(vetor_1d) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Cuidado com a Complexidade de Leitura

Não aninhe mais de duas regras em uma List Comprehension. Se a leitura ficar difícil, prefira usar laços for convencionais!

Tabela de Complexidade Assintótica de Operações em Listas

Conhecer o custo de cada operação em notação Big- O é essencial para projetar algoritmos eficientes em Algoritmos Avançados.

Operação / Método	Complexidade	Explicação Técnica
Acesso / Atribuição por índice ($l[i]$)	$O(1)$	Cálculo direto de endereço de memória
Adicionar ao final (<code>append</code>)	$O(1)$ amort.	Inserção no final sem deslocamento
Remover do final (<code>pop()</code>)	$O(1)$	Liberação do último elemento
Inserir no início/meio (<code>insert(i, x)</code>)	$O(n)$	Desloca $n - i$ elementos à direita
Remover do início/meio (<code>pop(0)</code> , <code>remove</code>)	$O(n)$	Desloca elementos à esquerda
Busca e Pertencimento (<code>x in l</code> , <code>index</code>)	$O(n)$	Varredura sequencial linear
Fatiamento ($l[a:b]$)	$O(k)$	Cópia de $k = b - a$ elementos
Ordenação (<code>sort()</code> , <code>sorted()</code>)	$O(n \log n)$	Algoritmo Timsort

Exercício Prático 1: Enunciado

Problema 1: Implementando Operações de TAD na Mão

Sem utilizar os métodos nativos `insert()`, `remove()` ou `pop()`, escreva funções Python para:

- ❶ `inserirnaposicao(lista, elemento, pos)` :
Insere um elemento deslocando os itens manualmente.

- ❶ `removerdaposicao(lista, pos)` :
Remove o elemento da posição informada deslocando os itens à esquerda.

Exercício Prático 1: Solução Comentada

```
1 def inserir_na_posicao(lista: list, elemento: any, pos: int) -> None:
2     # Adiciona um placeholder ao final para expandir o tamanho
3     lista.append(None)
4     # Desloca elementos uma posicao para a direita (de tras para frente)
5     for i in range(len(lista) - 1, pos, -1):
6         lista[i] = lista[i - 1]
7     lista[pos] = elemento
8
9 def remover_da_posicao(lista: list, pos: int) -> any:
10    valor_removido = lista[pos]
11    # Desloca elementos uma posicao para a esquerda
12    for i in range(pos, len(lista) - 1):
13        lista[i] = lista[i + 1]
14    # Remove a duplicata restante no final
15    del lista[-1]
16    return valor_removido
17
18 # Teste
19 dados = [10, 20, 30, 40]
20 inserir_na_posicao(dados, 99, 1) # [10, 99, 20, 30, 40]
```



Exercício Prático 2: Enunciado

Problema 2: Remoção de Duplicatas Preservando a Ordem

Dada uma lista de leituras numéricas de sensores com valores duplicados, escreva uma função que remova todas as duplicatas mantendo estritamente a primeira aparição de cada elemento.

- **Restrição:** Utilize apenas operações e verificações em listas.

Exercício Prático 2: Solução Comentada

```
1 def remover_duplicatas_ordenado(lista: list) -> list:
2     resultado = []
3
4     for item in lista:
5         # Verifica presença usando a busca sequencial na propria lista
6         if item not in resultado:
7             resultado.append(item)
8
9     return resultado
10
11 # Teste com dados brutos de sensores
12 leituras = [23.5, 18.0, 23.5, 40.1, 18.0, 25.2, 23.5]
13 sem_duplicatas = remover_duplicatas_ordenado(leituras)
14
15 print("Original:", leituras)
16 print("Limpo:      ", sem_duplicatas) # [23.5, 18.0, 40.1, 25.2]
```

Exercício Prático 3: Enunciado

Problema 3: Filtragem e Processamento Agrícola

Dadas três listas paralelas contendo o identificador do talhão, a área em hectares e a produção em sacas:

- 1 Calcule a produtividade de cada talhão (sacas/ha) e a produtividade média da propriedade.
- 2 Utilize List Comprehension para gerar uma lista com os identificadores dos talhões cuja produtividade foi superior à média calculada.

Exercício Prático 3: Solução Comentada

```
1 talhoes = ["T-01", "T-02", "T-03", "T-04"]
2 areas_ha = [100, 50, 120, 80]
3 producao_sacas = [7500, 3100, 9800, 4800]
4
5 # 1. Calculo de produtividade individual (sacas/ha) e media geral
6 produtividades = [p / a for a, p in zip(areas_ha, producao_sacas)]
7 media_propriedade = sum(produtividades) / len(produtividades)
8
9 # 2. Filtragem dos talhoes destaque usando List Comprehension
10 talhoes_destaque = [
11     t for t, prod in zip(talhoes, produtividades)
12     if prod > media_propriedade
13 ]
14
15 print(f"Produtividade Media Geral: {media_propriedade:.2f} sacas/ha")
16 print(f"Talhoes acima da media: {talhoes_destaque}")
```

Próximos Passos

Conteúdo da Próxima Aula (Aula 04)

- **Dicionários em Python (Estruturas de Chave-Valor)**
- Funcionamento de Tabelas Hash (Hashing, Colisões).
- Criação, métodos essenciais ('keys()', 'values()', 'items()', 'get()').
- Dictionary Comprehensions e Aplicação na Representação de Grafos (Listas de Adjacência).

Tarefa Complementar

Praticar os métodos de listas e resolver os exercícios da Lista 2 no repositório da disciplina.

Bons Estudos!

Dúvidas ou Perguntas?